data structures. You need to take extreme care when designing such data structures, because they're hard to get right, and the conditions that cause the design to fail may occur very rarely. We'll start by looking at what it means for data structures to be lock-free; then we'll move on to the reasons for using them before working through some examples and drawing out some general guidelines.

7.1. Definitions and consequences

Algorithms and data structures that use mutexes, condition variables, and futures to synchronize the data are called *blocking* data structures and algorithms. The application calls library functions that will suspend the execution of a thread until another thread performs an action. Such library calls are termed *blocking* calls because the thread can't progress past this point until the block is removed. Typically, the OS will suspend a blocked thread completely (and allocate its time slices to another thread) until it's *unblocked* by the appropriate action of another thread, whether that's unlocking a mutex, notifying a condition variable, or making a future *ready*.

Data structures and algorithms that don't use blocking library functions are said to be *nonblocking*. Not all such data structures are *lock-free*, though, so let's look at the various types of nonblocking data structures.

7.1.1. Types of nonblocking data structures

Back in chapter 5, we implemented a basic mutex using `std::atomic_flag` as a spin lock. The code is reproduced in the following listing.

Listing 7.1. Implementation of a spin-lock mutex using `std::atomic_flag`

```
class spinlock_mutex
{
    std::atomic_flag flag;
public:
    spinlock_mutex():
        flag(ATOMIC_FLAG_INIT)
    {}
```